Uso del Set en Java

En esta presentación, exploraremos la utilidad y flexibilidad del **Set** en Java, un tipo de colección que garantiza la unicidad de sus elementos, ofreciendo ventajas en diversas aplicaciones.

A **por Antonio Martín León**

Definición y características del Set

Definición

Un **Set** es una colección de elementos que no permite duplicados. Los elementos se almacenan en un orden no definido, a menos que se especifique una implementación ordenada.

Características

- No permite duplicados.
- Orden no definido (a menos que se ordene).
- Soporta operaciones como agregar, eliminar, iterar.

Implementaciones del Set

HashSet

Implementación basada en una tabla hash. Ofrece acceso rápido y una gestión eficiente de elementos.

EnumSet

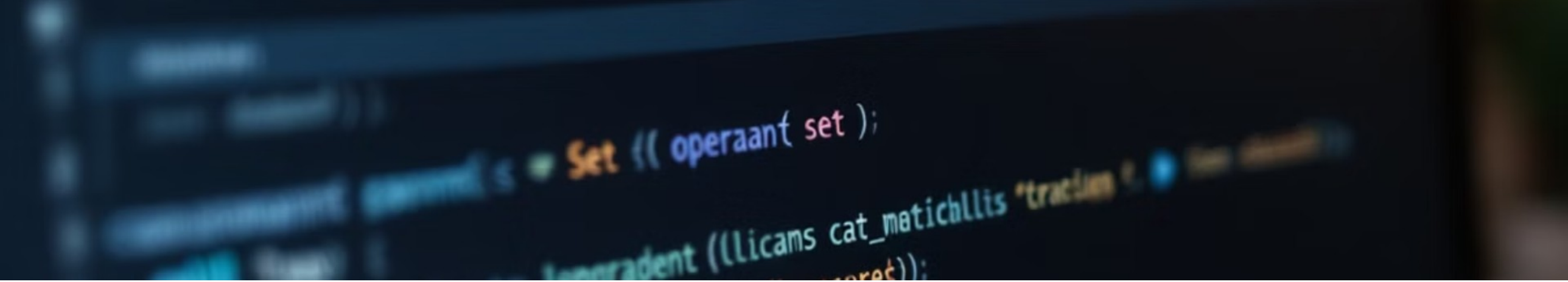
Especializado para trabajar con enumeraciones. Optimizado para eficiencia y uso con tipos de datos enumerados.

LinkedHashSet

Mantiene el orden de inserción de los elementos. Combina la eficiencia de HashSet con el orden de LinkedHashSet.

TreeSet

Implementación ordenada basada en un árbol binario. Permite la ordenación natural o personalizada de elementos.



Operaciones básicas en el Set

1

2

3

4

Agregar

Agrega un elemento al set. Si el elemento ya existe, no se añade.

Acceder

Verifica si un elemento está presente en el set. No permite acceder a elementos por índice.

Eliminar

Elimina un elemento del set. Si el elemento no está presente, no se produce ningún efecto.

Iterar

Recorre los elementos del set. El orden de iteración no está definido, a menos que se use una implementación ordenada.

Aplicaciones del Set

Eliminación de duplicados

El Set es ideal para eliminar duplicados de una colección de datos, asegurando la unicidad de cada elemento.

Ordenamiento

Con implementaciones como TreeSet, los elementos se ordenan de forma natural o personalizada, facilitando el acceso y la búsqueda.

Gestión de permisos

El Set es útil para gestionar permisos de usuarios o roles, asegurando que cada usuario tenga acceso a los recursos correspondientes.

Uso de HashSet

Características

- Acceso rápido y eficiente.
- Implementación basada en tablas hash.
- No mantiene el orden de inserción.

Ejemplo

```
Set nombres = new HashSet<>();
nombres.add("Ana");
nombres.add("Pedro");
nombres.add("Ana"); // No se añade
System.out.println(nombres); // [Pedro, Ana]
```

Uso de EnumSet

Características

• Especializado para trabajar con enumeraciones.
• Optimizado para eficiencia y uso con tipos de datos enumerados.

• No admite la adición de elementos que no sean de la enumeración.

Ejemplo

```
enum Dias { LUNES, MARTES, MIERCOLES }
Set diasLaborables = EnumSet.of(Dias.LUNES,
Dias.MARTES, Dias.MIERCOLES);
System.out.println(diasLaborables); // [LUNES,
MARTES, MIERCOLES]
```

Uso de LinkedHashSet

Características

- Mantiene el orden de inserción de los elementos.
- Combina la eficiencia de HashSet con el orden de LinkedHashSet.
- Permite un recorrido ordenado de los elementos.

Ejemplo

```
Set ciudades = new LinkedHashSet<>();
ciudades.add("Madrid");
ciudades.add("Barcelona");
ciudades.add("Sevilla");
System.out.println(ciudades); // [Madrid,
Barcelona, Sevilla]
```

Uso de TreeSet

Características

• Implementación ordenada basada en un árbol binario.
• Permite la ordenación natural o personalizada de elementos.

• Ofrece operaciones de búsqueda y ordenamiento eficientes.

Ejemplo

```
Set numeros = new TreeSet<>();
numeros.add(5);
numeros.add(1);
numeros.add(3);
System.out.println(numeros); // [1, 3, 5]
```

Conclusión y consideraciones finales

El **Set** en Java es una herramienta fundamental para gestionar colecciones de elementos únicos. Su flexibilidad, eficiencia y diversas implementaciones lo convierten en una opción ideal para diversas aplicaciones, desde la eliminación de duplicados hasta el ordenamiento y la gestión de permisos.